FinSecure Txn API – Design Document

Jonathan Wylliem
GitHub: `https://github.com/YOUR_GITHUB_USER/finsecure-txn-api`
Docker Hub: `https://hub.docker.com/r/YOUR_DOCKERHUB_USER/finsecure-txn-api`
Demo video: `DEMO_VIDEO_URL`

1. Introduction

FinSecure Pay want to deploy their transaction validation API faster and with less manual steps. In this project I build a small Spring Boot API with a `/health` and a `/validate` endpoint, and a Jenkins pipeline that build, test, scan and deploy it to Amazon EKS. Prometheus and Grafana is used for monitoring and alert is sent by email.

2. Architecture

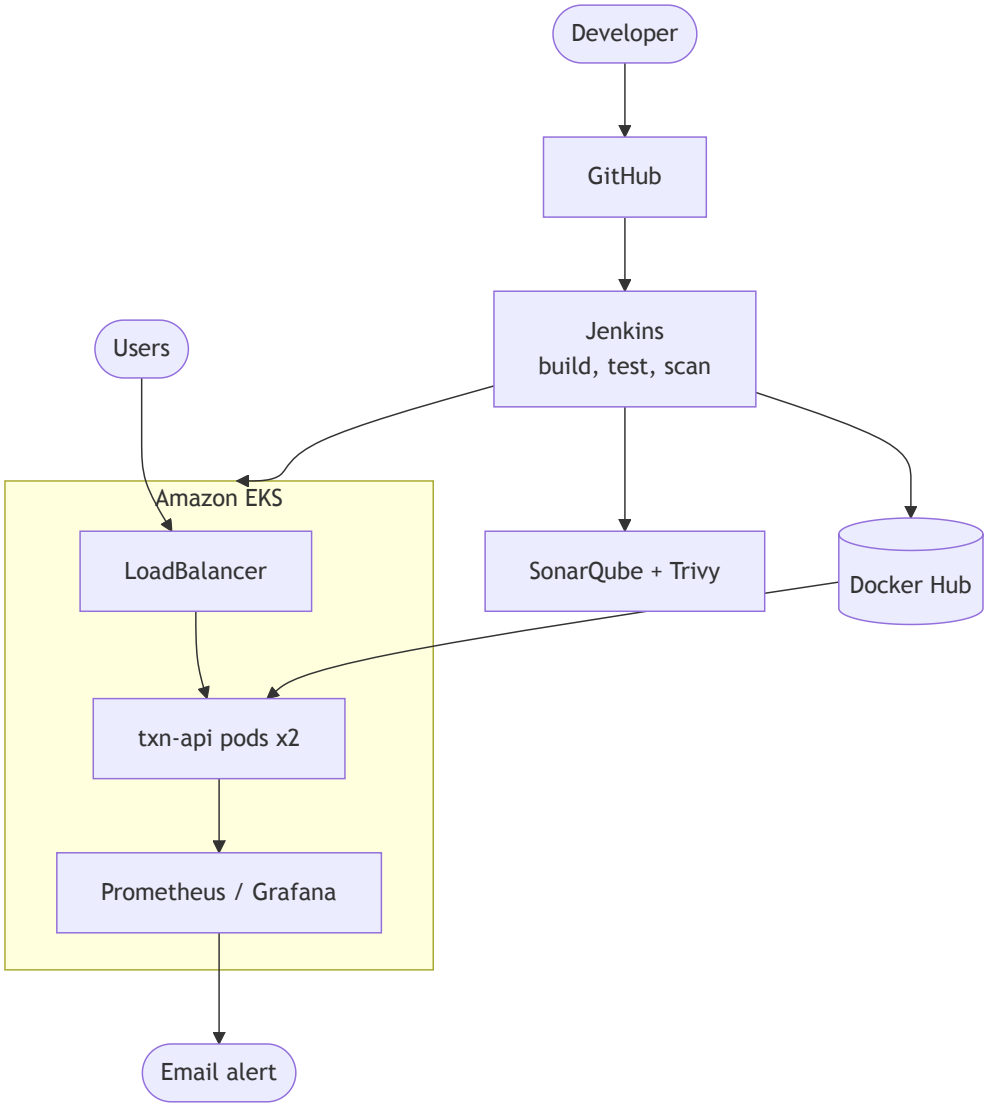


Figure 1: Architecture

3. Tools used

Area	Tool	Reason
Source control	Git, GitHub	recommended in brief

Application	Spring Boot, Maven, JUnit 5	recommended in brief, I used Java before
CI/CD	Jenkins with Jenkinsfile	recommended, run in Docker on my laptop and configured with JCasC
Container	Docker, Docker Hub	free registry
Kubernetes	Amazon EKS (eksctl)	we used AWS in earlier modules, eksctl create cluster from one file
Security	SonarQube, Trivy	Trivy is used instead of OWASP Dependency-Check because it is faster and also can scan the image and secrets
Monitoring	Prometheus, Grafana, Alertmanager	kube-prometheus-stack helm chart install all in one go
Logging	`kubectl logs` with JSON (ECS) format, `jq`	simple, enough for a small app, JSON make it easy to filter by level or message
Notification	Jenkins Email Extension, Gmail SMTP	recommended

4. Design decisions

- EKS was chosen because AWS manage the control plane, so I only take care of the nodes. 2 nodes so if one node down the app still running.
- The app always have at least 2 pods. A HorizontalPodAutoscaler scale it from 2 to 4 pods when CPU is above 70% of the request. The metrics-server add-on give the CPU numbers.
- A PodDisruptionBudget keep at least 1 pod running when a node is drained or upgraded.
- Each pod request 250m CPU and 384Mi memory, limit 1 CPU and 512Mi. The numbers come from Grafana, the app use around 200-250Mi when running, so there is some space.
- Rolling update use maxUnavailable 0, so the old pod only removed after the new pod is ready (no downtime).
- A startup probe give the JVM up to 2 minutes to start, then readiness and liveness probe (Spring Boot actuator) take over.
- Pods run as non-root user 10001, no privilege escalation and all Linux capabilities dropped.
- The service is a LoadBalancer so AWS create a load balancer with a public address for testing.
- The max amount and currencies are in a ConfigMap so they can change without rebuild the image.
- Image tag is version + build number + git commit, e.g. `1.0.0-5-a1b2c3d`.
- Dockerfile is multi-stage so the final image only have the JRE and run as non-root user.
- Nodes are t4g.medium (arm64) because my Mac build arm64 images.
- Jenkins check GitHub every 2 minutes (poll SCM) because it run on my laptop and cannot receive webhook.

5. Security

- SonarQube quality gate, Trivy dependency scan, Trivy secret scan and Trivy image scan run in the pipeline. HIGH or CRITICAL vulnerability fail the pipeline so the image is not pushed.
- To test, I added `commons-text 1.9` which have CVE-2022-42889 (critical). The pipeline was blocked. After upgrade to a fixed version it pass.
- Passwords and tokens are stored in Jenkins credentials (loaded from a `.env` file not committed to git). The API key of the app is a Kubernetes Secret.
- Jenkins use a separate service account that only can deploy in the `finsecure` namespace.

6. Recovery

- If a pod is deleted or crash, Kubernetes create a new one automatically (shown in demo).

- If the new version rollout fail, Jenkins run `kubectl rollout undo`. We can also rollback manually with the `ROLLBACK_TO` parameter.
- If a test fail, the pipeline stop and nothing is deployed.

7. Monitoring and logging

- Grafana dashboard have 6 panels: validations per second (valid/invalid), /validate p95 latency, ready pods vs HPA desired, pod restarts, CPU and memory.
- 3 alerts go to email by Alertmanager: many invalid transactions (warning), less than 2 ready pods (critical) and p95 latency above 500ms (warning).
- Logs are JSON so we can filter with jq, e.g. only the invalid transaction or only ERROR. Steps are in the README.

8. AI use case: AI for alert analysis

Problem: when an alert come in, the engineer need time to check logs and metrics to find the cause. AI (a large language model) can read the logs and alert and give a summary of the possible cause faster than a human.

Inputs: the alert from Alertmanager, recent pod logs, metrics from Prometheus and the recent deployments.

Benefits: faster troubleshooting, less time to recover, help junior engineers.

Risks: AI can give wrong answer, so human must check before taking action. Logs may contain customer data so it must be masked before sending to AI. There is also cost for using the AI service.

Integration: Alertmanager can send a webhook to a small service which collect the logs and call the AI API, then send the summary by email together with the alert.

9. Lessons learned and future improvements

- Setting up Jenkins took the most time, a bit sian at first, but using JCasC helped to rebuild it quickly.
- A test found a bug where a missing currency crash the validator, so unit tests are useful.
- Future: centralised logging (EFK or CloudWatch), HTTPS with an Ingress, and GitOps with Argo CD.